



EMERITUS

Learn. From the world's best.

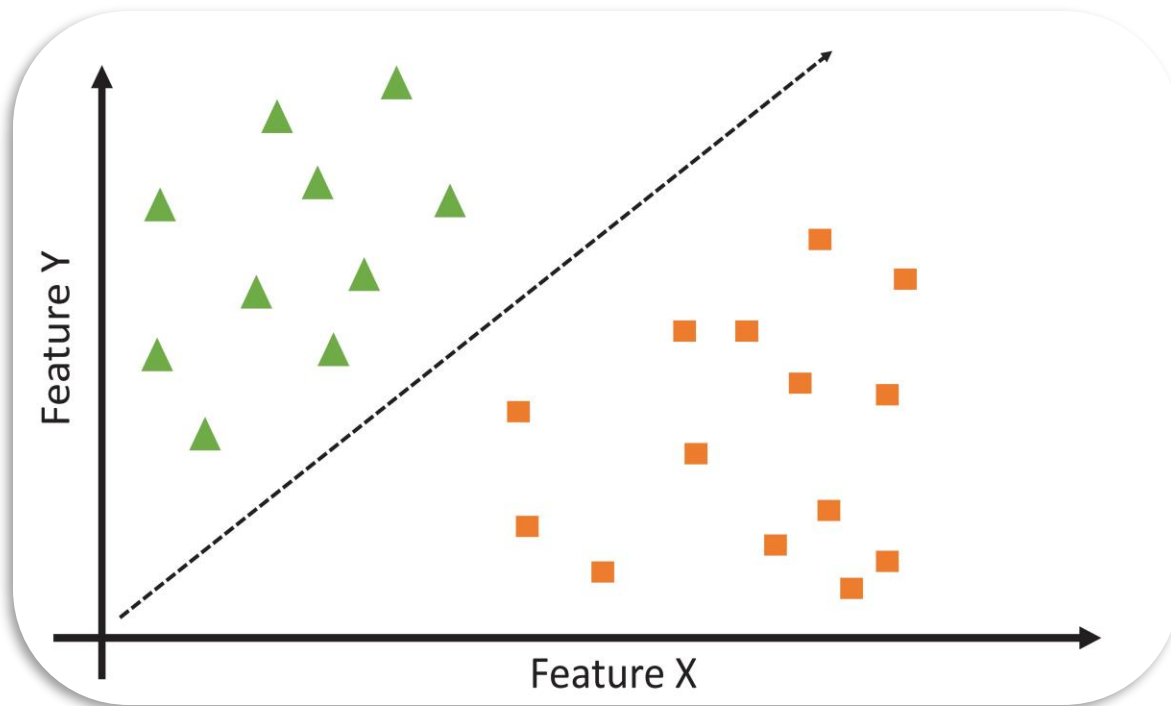
Python Fundamentals – Part II (Contd)



Matplotlib

Introduction to Matplotlib

Powerful Data Visualisation



- ➡ A Python library for creating charts and graphs
- ➡ Supports static, animated and interactive plots
- ➡ Ideal for exploring and presenting data

Sales Data Example

Loading Data

First, let's load the sales data from a CSV file

```
import pandas as pd  
  
# Load the sales data  
sales_data = pd.read_csv('sales_data.csv')  
print(sales_data)
```

Product	Price	Sales	Cost	Region
Smartphone	600	500	400	South
Monitor	300	150	100	West
Smartphone	650	600	450	North
Mouse	50	200	30	West
Monitor	350	200	150	East
Headphones	170	140	90	West
Smartphone	600	500	400	South
Monitor	300	150	100	West
Smartphone	650	600	450	North
Mouse	50	200	30	West
Monitor	350	200	150	East
Headphones	170	140	90	West

Line Plot: Sales by Region

Objective:

Visualise the relationship between **Sales** and **Region** using a line chart



Line Plot: Sales by Region

Code

```
# we will summarize data and plot it  
sales_data_agg = sales_data.groupby('Region')['Sales'].sum()  
plt.figure(figsize=(8, 6))  
plt.plot(sales_data_agg.index, sales_data_agg, marker='o', linestyle='-',  
color='b')  
plt.title('Sales vs Price')  
plt.xlabel('Region')  
plt.ylabel('Sales')  
plt.grid(True)  
plt.show()
```

Output

- A line chart displaying **Sales trends across Regions**

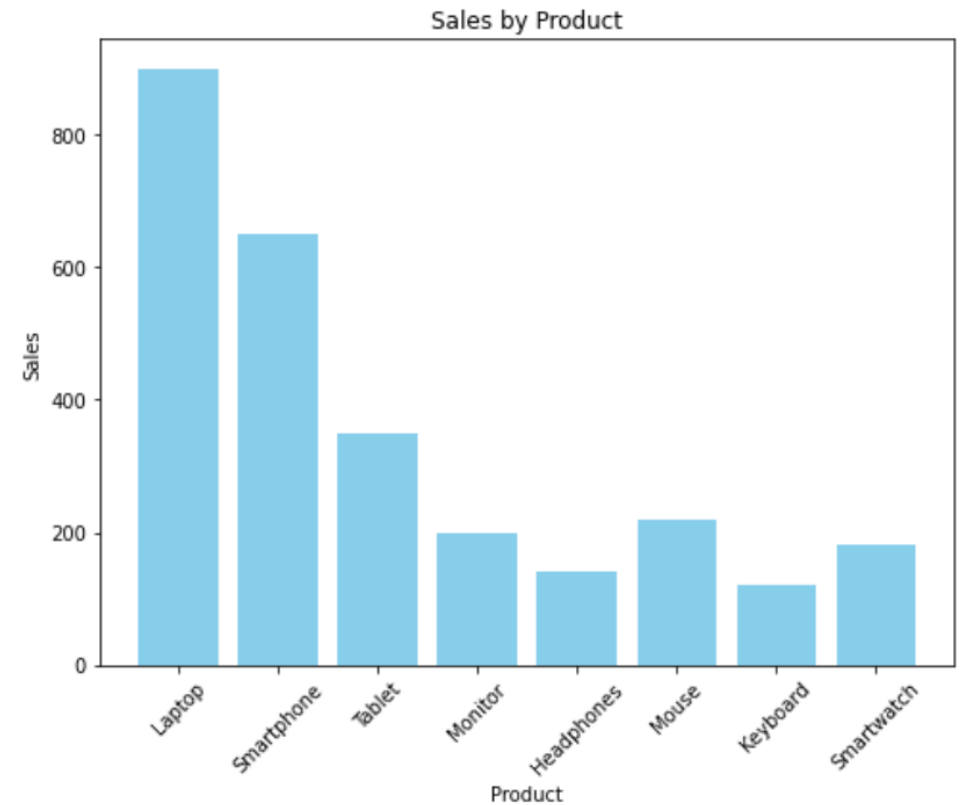
Bar Plot: Sales by Product

Objective

Compare **Sales** across different **Products** using a bar chart

Code

```
# Bar plot: Sales by Product  
plt.figure(figsize=(8, 6))  
plt.bar(sales_data['Product'],  
sales_data['Sales'], color='skyblue')  
plt.title('Sales by Product')  
plt.xlabel('Product')  
plt.ylabel('Sales')  
plt.xticks(rotation=45)  
plt.show()
```



Output

A bar chart that displays the sales for each product

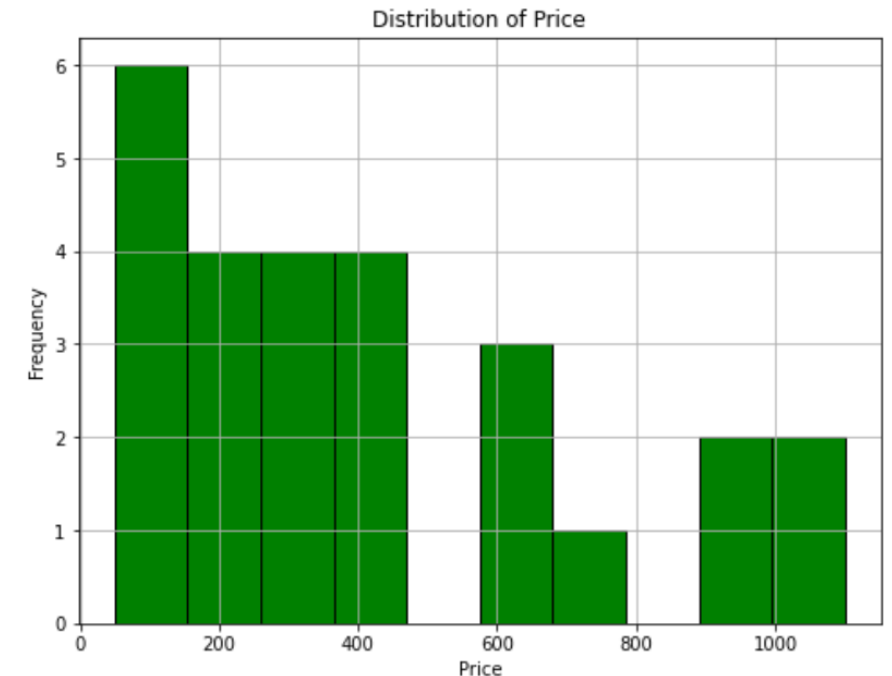
Histogram: Distribution of Prices

Objective

Plot a histogram to see the distribution of Price

Code

```
# Histogram: Distribution of Price  
plt.figure(figsize=(8, 6))  
plt.hist(sales_data['Price'], bins=10, color='green',  
edgecolor='black')  
plt.title('Distribution of Price')  
plt.xlabel('Price')  
plt.ylabel('Frequency')  
plt.grid(True)  
plt.show()
```



Output

A histogram showing the frequency distribution of Price values

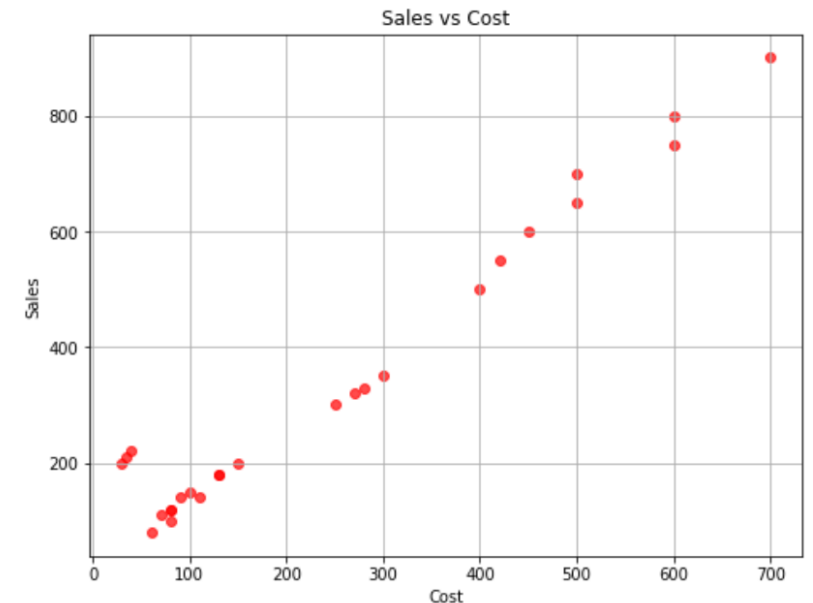
Scatter Plot: Sales vs Cost

Objective

- Analyse the relationship between **Sales** and **Cost** using a scatter plot

Code

```
# Scatter plot: Sales vs Cost  
plt.figure(figsize=(8, 6))  
plt.scatter(sales_data['Cost'],  
sales_data['Sales'], color='red',  
alpha=0.7)  
plt.title('Sales vs Cost')  
plt.xlabel('Cost')  
plt.ylabel('Sales')  
plt.grid(True)  
plt.show()
```



Output

- A **scatter plot** showing the correlation between **Sales** and **Cost**

Pie Chart: Sales by Region

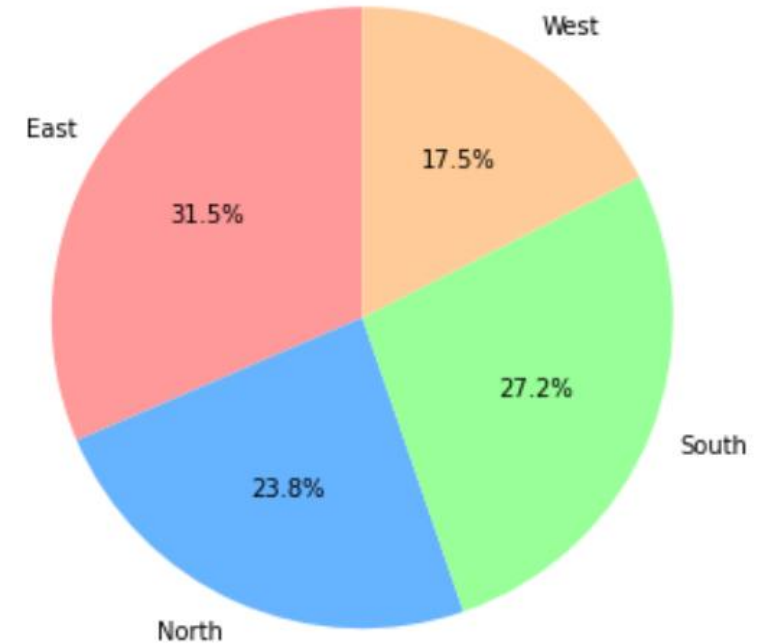
Objective

- Show the percentage of **Sales** in each **Region** using a pie chart

Data Preparation

- Group **Sales data** by **Region**.

Sales Distribution by Region



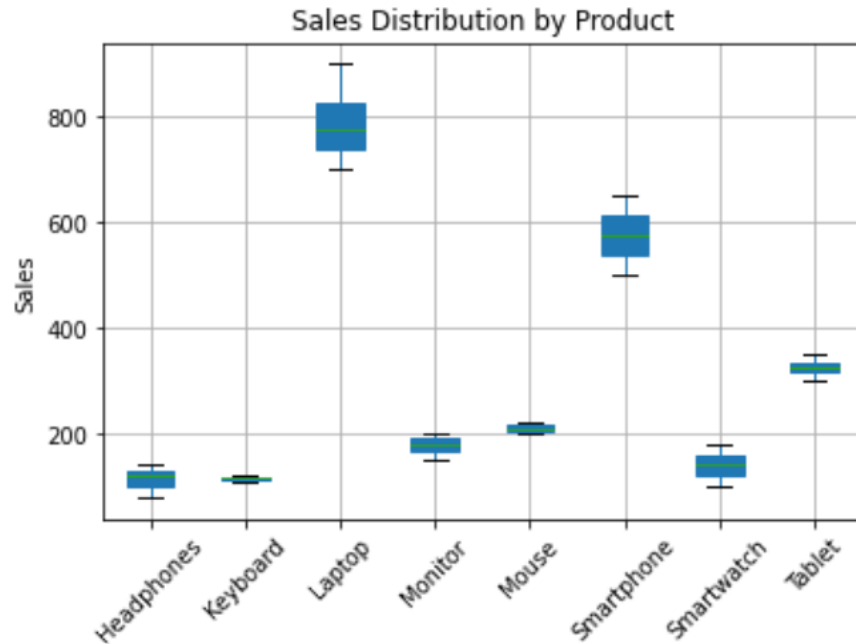
Pie Chart: Sales by Region

Visualisation

- Generate a **pie chart** with percentage labels

```
# Pie chart: Sales by Region  
sales_by_region = sales_data.groupby('Region')['Sales'].sum()  
plt.figure(figsize=(8, 6))  
plt.pie(sales_by_region, labels=sales_by_region.index,  
autopct='%1.1f%%', startangle=90,  
colors=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99'])  
plt.title('Sales Distribution by Region')  
plt.show()
```

Box Plot: Sales by Product



Objective

Display the Sales distribution for each Product using a box plot

Box Plot: Sales by Product

Code

```
# Box plot: Sales by Product  
plt.figure(figsize=(8, 6))  
sales_data.boxplot(column='Sales', by='Product', patch_artist=True)  
plt.title('Sales Distribution by Product')  
plt.suptitle("")  
plt.xlabel('Product')  
plt.ylabel('Sales')  
plt.grid(True)  
plt.show()
```

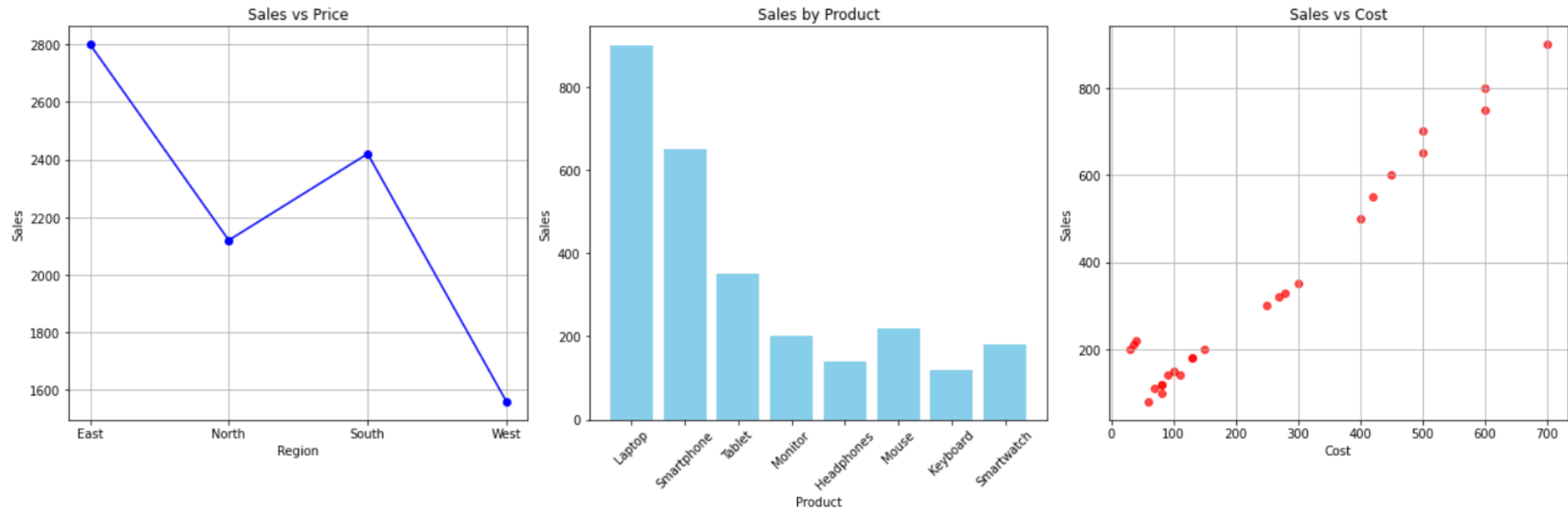
Output

- A **box plot** highlighting **Sales distribution and outliers** for each **Product**

Subplots: Multiple Charts in One Figure

Objective

- Display **multiple visualisations** in a **single figure**



Subplots: Multiple Charts in One Figure

```
# Subplots: Line Plot, Bar Plot, and Scatter Plot  
fig, axs = plt.subplots(1, 3, figsize=(18, 6))
```

```
# Line plot
```

```
axs[0].plot(sales_data['Price'], sales_data['Sales'], marker='o', linestyle='-', color='b')  
axs[0].set_title('Sales vs Price')  
axs[0].set_xlabel('Price')  
axs[0].set_ylabel('Sales')  
axs[0].grid(True)
```

```
# Bar plot
```

```
axs[1].bar(sales_data['Product'], sales_data['Sales'], color='skyblue')  
axs[1].set_title('Sales by Product')  
axs[1].set_xlabel('Product')  
axs[1].set_ylabel('Sales')  
axs[1].tick_params(axis='x', rotation=45)
```


Subplots: Multiple Charts in One Figure

```
# Scatter plot  
axs[2].scatter(sales_data['Cost'], sales_data['Sales'], color='red', alpha=0.7)  
axs[2].set_title('Sales vs Cost')  
axs[2].set_xlabel('Cost')  
axs[2].set_ylabel('Sales')  
axs[2].grid(True)  
  
plt.tight_layout()  
plt.show()
```

Output:

A single figure with three plots: line plot, bar plot and scatter plot

Introduction to Seaborn

Seaborn: A Powerful Visualisation Library

What is Seaborn?

A Python library for creating statistical visualisations

Built on Matplotlib

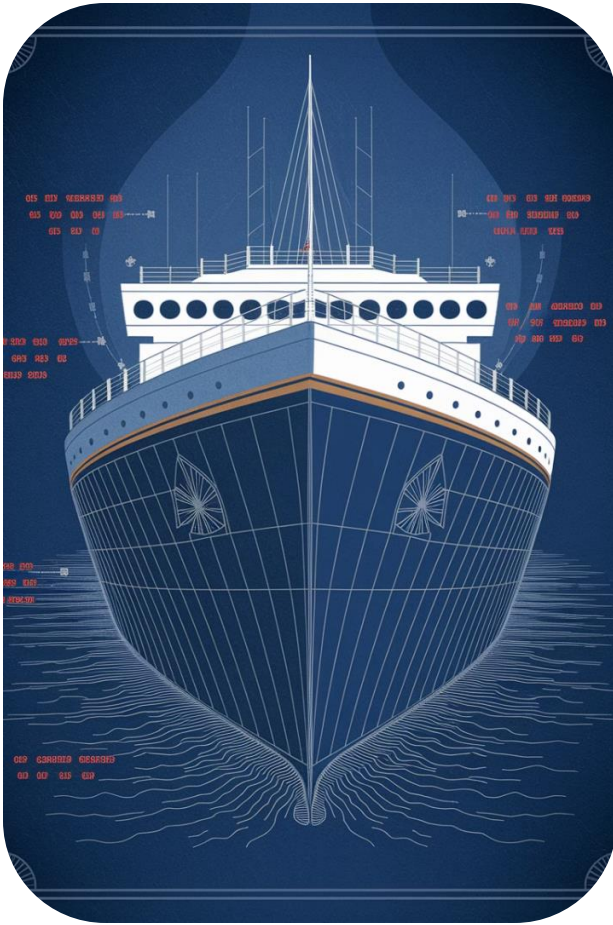
Provides a high-level interface for drawing attractive and informative graphics

Why Use Seaborn?

- Simplifies complex visualisations
- Automatically manages aesthetics and colours
- Works seamlessly with pandas DataFrames



Loading the Titanic Dataset



```
import seaborn as sns
```

```
import pandas as pd
```

```
# Load Titanic dataset
```

```
titanic = sns.load_dataset("titanic")
```

```
# Display first few rows
```

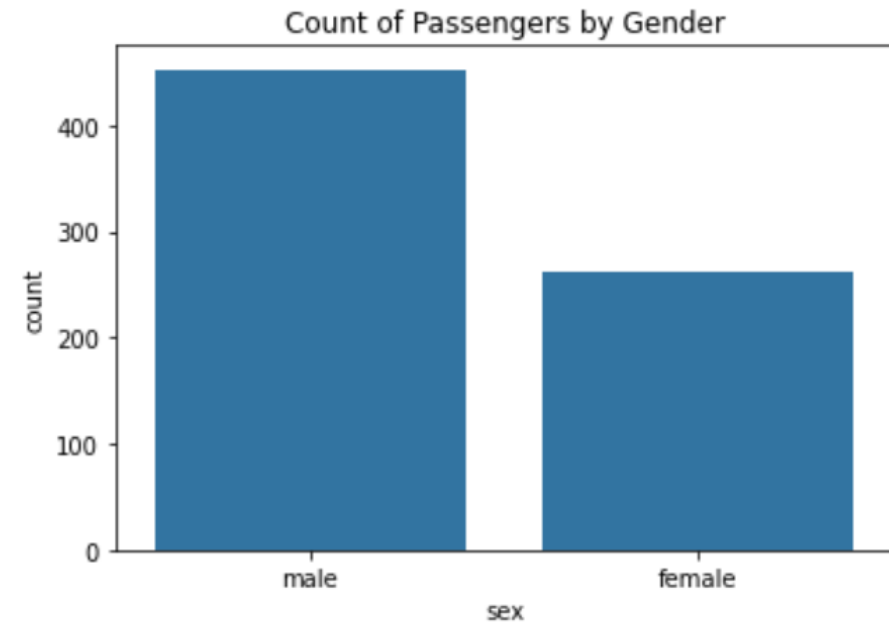
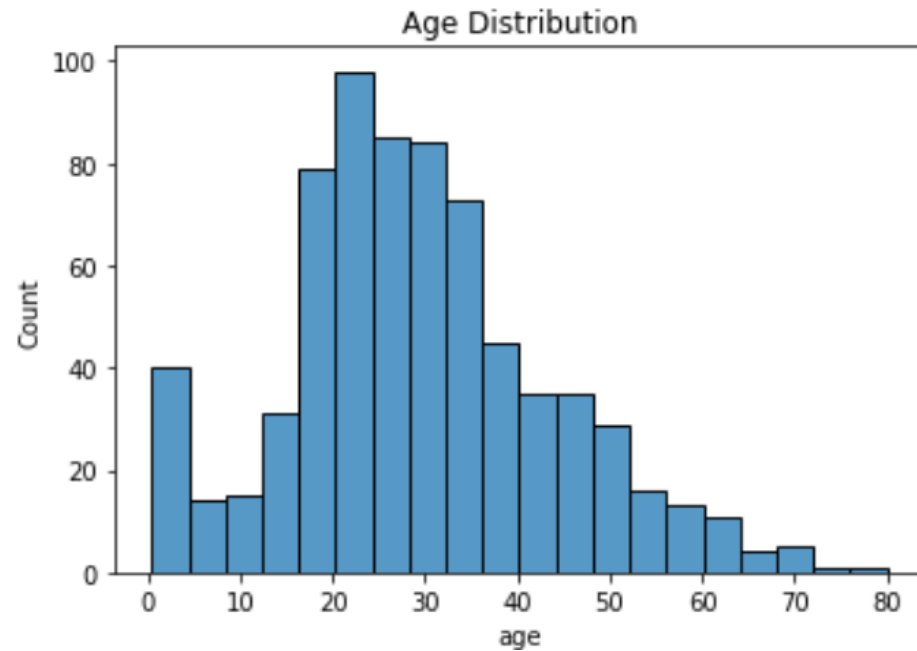
```
print(titanic.head())
```

Univariate Analysis

Examining a Single Variable

Definition

Analysing one variable at a time



Univariate Analysis

Examining a Single Variable

Key Plots:

Histogram: Shows the distribution of numerical data

Count Plot: Displays the frequency of categorical data

```
import matplotlib.pyplot as plt  
# Histogram for age distribution  
sns.histplot(data=titanic, x="age", kde=True)  
plt.title("Age Distribution")  
plt.show()
```

```
# Count plot for gender  
sns.countplot(data=titanic, x="sex")  
plt.title("Count of Passengers by Gender")  
plt.show()
```

Bivariate Analysis

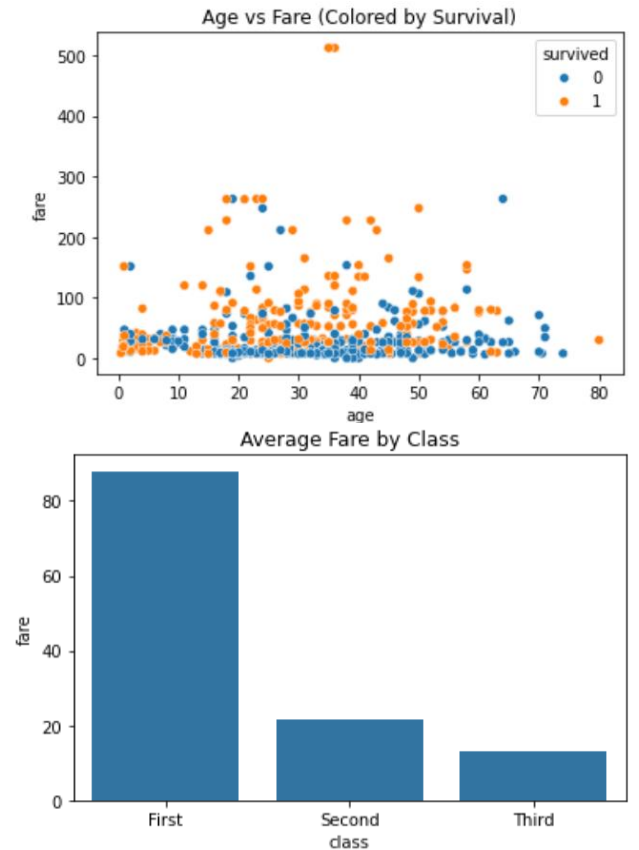
Exploring Relationships Between Two Variables

What It Means

- Examines how two variables interact

Key Visuals

- **Scatter Plot** – Relationship between two numerical values
- **Bar Chart** – Average values across categories
- **Box Plot** – Distribution patterns within groups

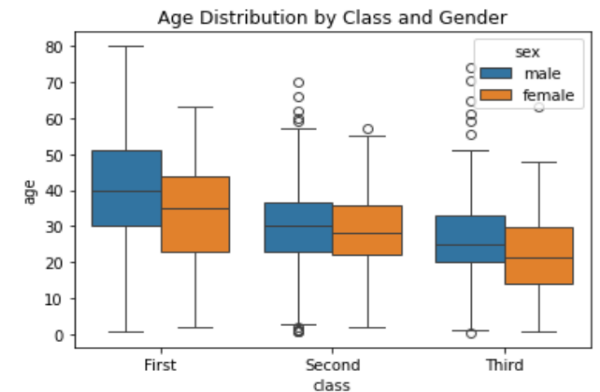


Bivariate Analysis

Exploring Relationships Between Two Variables

Code Example

```
# Scatter plot: Age vs Fare  
sns.scatterplot(data=titanic, x="age", y="fare",  
hue="survived")  
plt.title("Age vs Fare (Colored by Survival)")  
plt.show()  
  
# Bar plot: Average fare by class  
sns.barplot(data=titanic, x="class", y="fare", ci=None)  
plt.title("Average Fare by Class")  
plt.show()  
  
# Box plot: Age distribution by class  
sns.boxplot(data=titanic, x="class", y="age", hue="sex")  
plt.title("Age Distribution by Class and Gender")  
plt.show()
```



Multivariate Analysis

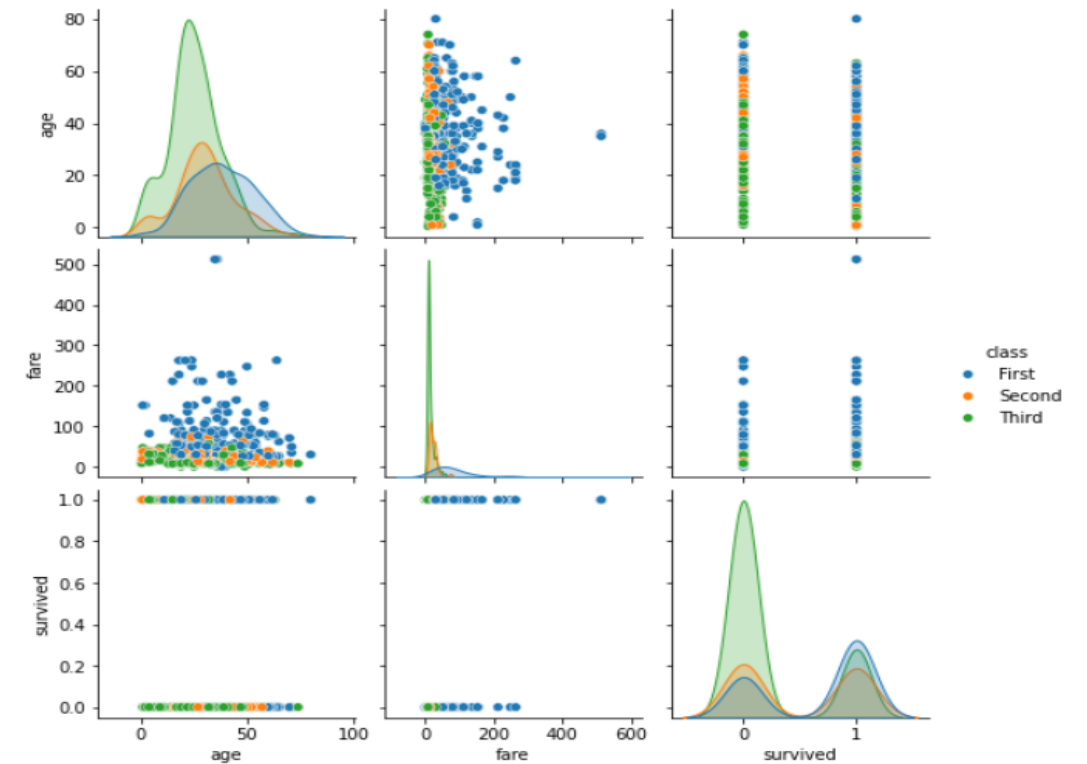
Exploring Relationships Among Multiple Variables

What It Means

- Examines how three or more variables interact

Key Visuals

- **Heatmap** – Correlation between variables
- **Pair Plot** – Pairwise relationships in data



Multivariate Analysis

Exploring Relationships Among Multiple Variables

Code Example

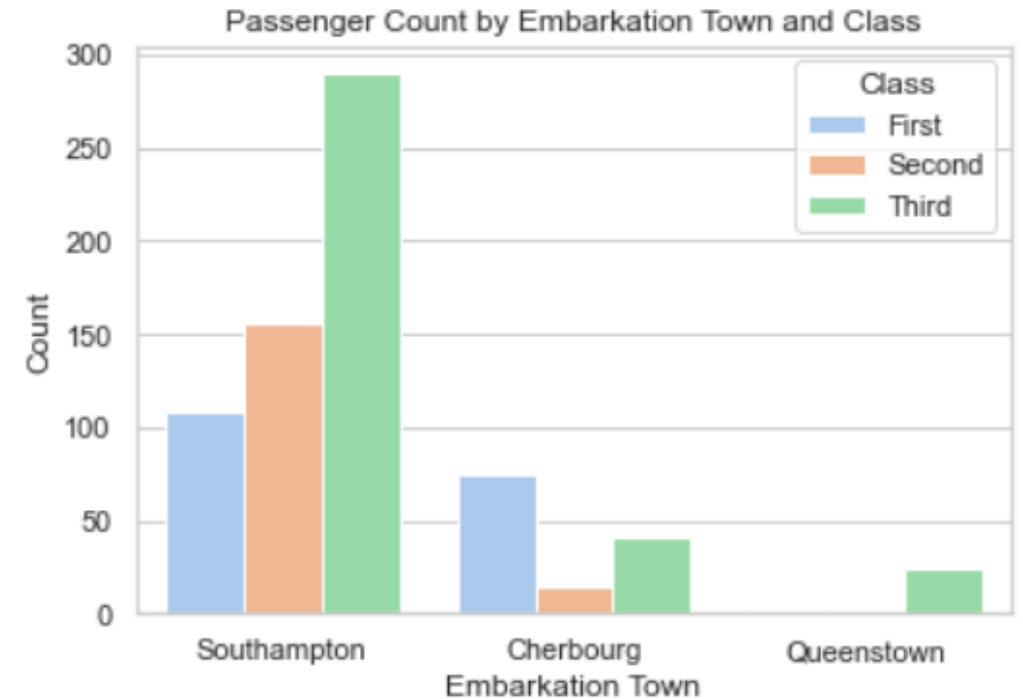
```
# Heatmap: Correlation between numerical variables  
corr = titanic.corr(numeric_only=True)  
sns.heatmap(corr, annot=True, cmap="coolwarm")  
plt.title("Correlation Heatmap")  
plt.show()  
  
# Pair plot for selected variables  
sns.pairplot(titanic, vars=["age", "fare", "survived"], hue="class")  
plt.show()
```

Customising Visualisations

Enhancing Charts For Better Insights

Key Enhancements

- **Annotations** – Add extra information
- **Colours** – Adjust palettes for clarity
- **Styles** – Change themes for better readability



Customising Visualisations

Enhancing Charts For Better Insights

```
# Changing the style and palette  
sns.set_theme(style="whitegrid", palette="pastel")  
  
# Customized count plot  
sns.countplot(data=titanic, x="embark_town", hue="class")  
plt.title("Passenger Count by Embarkation Town and Class")  
plt.xlabel("Embarkation Town")  
plt.ylabel("Count")  
plt.legend(title="Class")  
plt.show()
```

Faceted Visualisations

Multiple Subplots For Different Categories

Key Method

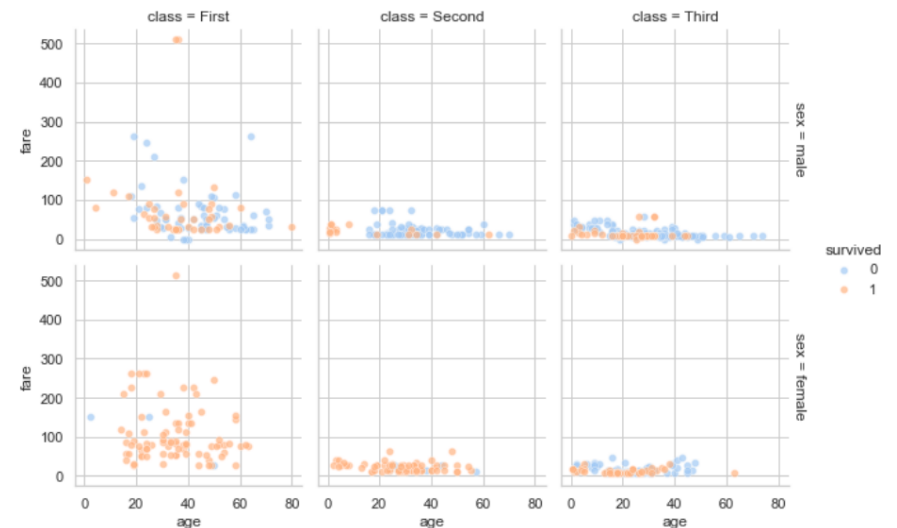
- **FacetGrid** – Creates grouped plots based on categories

Code Example

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load Titanic dataset
titanic = sns.load_dataset("titanic")

# Faceted grid: Survival by age and sex
g = sns.FacetGrid(titanic, col="sex",
                  row="class", hue="survived",
                  margin_titles=True)
g.map(sns.scatterplot, "age", "fare",
      alpha=0.7)
g.add_legend()
plt.show()
```



Relational Plots

Key Insights

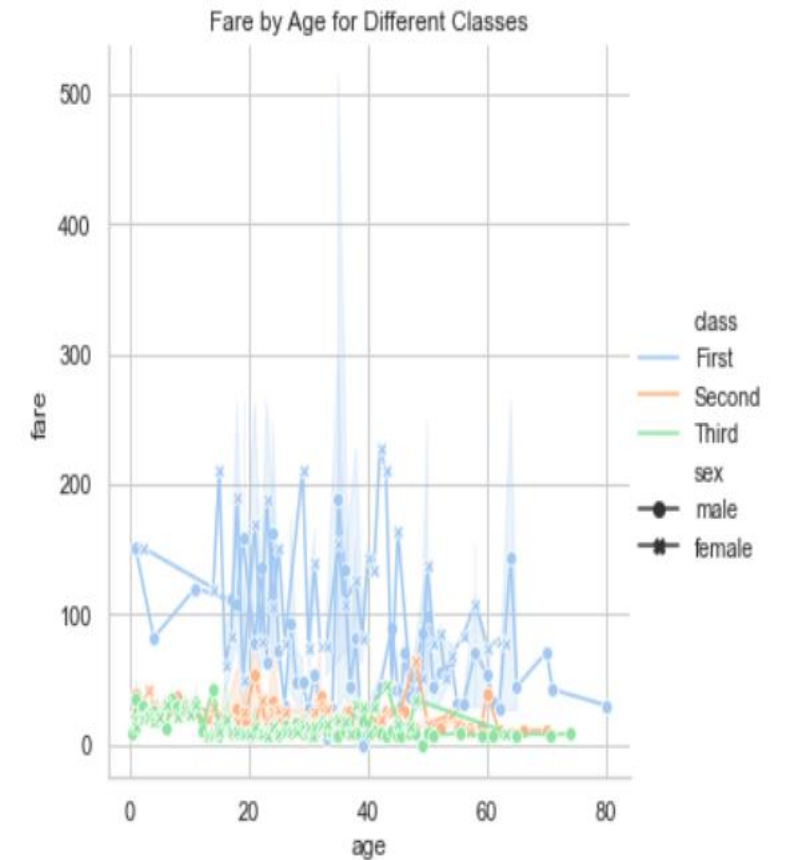
Segments in Plots – Show relationships between data points

Example – Line segments connecting data

Method – `sns.relplot` for relational plots

Code Example

```
# Relational plot with line segments for fare by age
sns.relplot(
    data=titanic, x="age", y="fare", kind="line",
    hue="class", style="sex", markers=True, dashes=False
)
plt.title("Fare by Age for Different Classes")
plt.show()
```



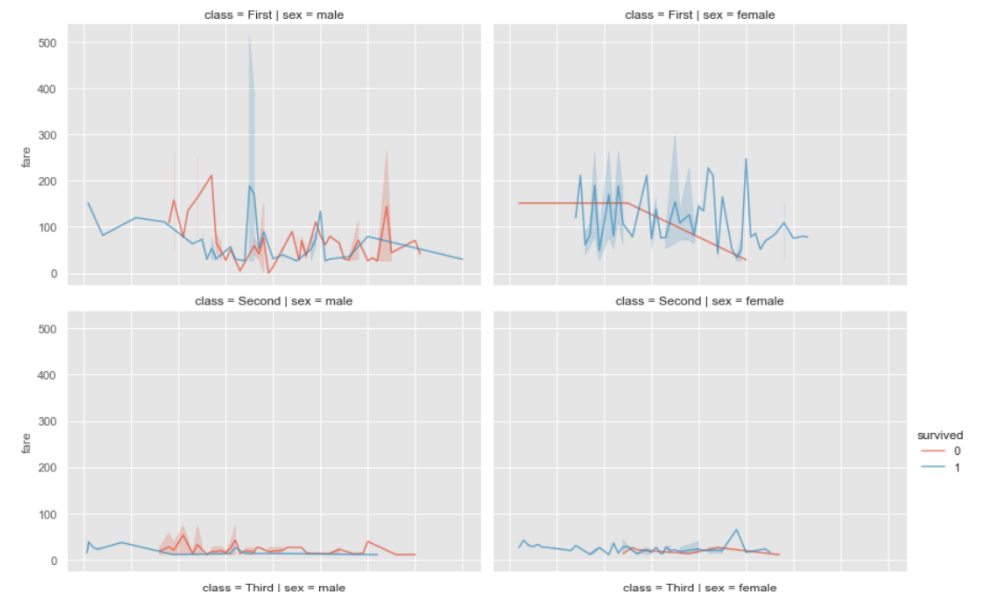
Combining Panels and Segments

Key Insights

Complex Grids : Display multiple variables together

Methods : Use *sns.relplot* or *FacetGrid* for layered visualisations

```
# FacetGrid with segments for survival and fare  
g = sns.FacetGrid(titanic, col="sex", row="class",  
hue="survived", height=4, aspect=1.5)  
g.map(sns.lineplot, "age", "fare", alpha=0.7)  
g.add_legend()  
plt.show()
```



Advanced Categorical Plots

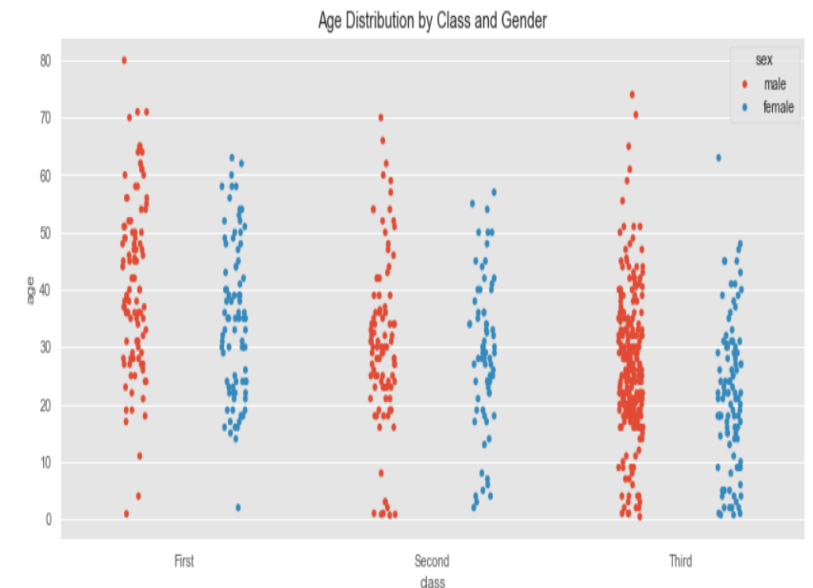
Enhancing Data Distribution Clarity

Key Insights

- **Strip Plot with Jitter** – Spreads points for better visibility
- **Swarm Plot** – Prevents overlapping data points

```
# Strip plot with jitter  
sns.stripplot(data=titanic, x="class", y="age", jitter=True,  
hue="sex", dodge=True)  
plt.title("Age Distribution by Class and Gender")  
plt.show()
```

```
# Swarm plot  
sns.swarmplot(data=titanic, x="class", y="fare",  
hue="sex", dodge=True)  
plt.title("Fare Distribution by Class and Gender")  
plt.show()
```



Best Practices

Key Tips

“

Understand the Data: Explore before plotting

Choose the Right Plot: Match visuals to data type

Keep It Clear: Avoid clutter and overcomplication

Use Meaningful Labels: Titles and axes should be informative

Select Accessible Colours:
Ensure readability for all users

”



Simple Python Deployment Demo

Introduction

“

Deploying a basic Python web API



Simple Deployment – Turn a Python app into a web API

Versatile Approach – Use the same method for ML models

”



Setting Up the Environment

Install Flask (Lightweight Web Framework)

```
pip install flask
```



Building a Simple Web API

Flask-Based API for Basic Operations

- The `/` route displays a welcome message
- The `/add` route accepts two numbers as query parameters and returns their sum
- This approach can also be used to deploy machine learning models for predictions

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to the Python Deployment Demo!"

@app.route('/add', methods=['GET'])
def add_numbers():
    a = request.args.get('a', type=int)
    b = request.args.get('b', type=int)
    result = {'sum': a + b}
    return jsonify(result)

if __name__ == '__main__':
    app.run(debug=True)
```

Running the API Locally

Start the API

Run the command:

```
python app.py
```

Access the API

The API will be available at:

<http://127.0.0.1:5000/>

Test in a Browser

- **<http://127.0.0.1:5000/>** → Displays a **welcome message**.
- **<http://127.0.0.1:5000/add?a=5&b=10>** → Returns {"sum": 15}

Deploying Flask with Gunicorn

Production-Ready

Install Gunicorn

Run the command:

```
pip install gunicorn
```

Start Flask with Gunicorn

Run the command

```
gunicorn -w 4 -b 0.0.0.0:5000
```

Key Details

- Runs the Flask app with **4 worker processes**.
- Access it at **<http://localhost:5000/>**



Exploratory Data Analysis (EDA)

EDA and Data Preprocessing

Getting Started with Data Preprocessing

What is Data Preprocessing?

A key step in data analysis and machine learning to improve data quality and model performance



Main Steps:

- Handle missing data
- Identify and treat outliers
- Encode categorical features
- Scale features
- Engineer new features

Demo: Titanic dataset

Handling Missing Data

Why It's Important

- Missing data can affect analysis and model accuracy
- Imputation helps manage missing values effectively

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load Titanic dataset
df = sns.load_dataset('titanic')

# Check for missing values
print(df.isnull().sum())

# Fill missing numerical values with median
df['age'].fillna(df['age'].median(), inplace=True)
df['fare'].fillna(df['fare'].median(), inplace=True)

# Fill missing categorical values with mode
df['embark_town'].fillna(df['embark_town'].mode()[0], inplace=True)

# Verify missing values
print(df.isnull().sum())
```


Handling Outliers

Why Remove Outliers?

- Distort data distribution
- Reduce model accuracy

Remove Outliers (IQR Method)

```
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
```

```
# Define the bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
# Remove outliers
df = df[(df['fare'] >= lower_bound) & (df['fare'] <= upper_bound)]
```

Visualise Outliers (Boxplot)

```
plt.figure(figsize=(8,5))
sns.boxplot(x=df['fare'])
plt.title("Boxplot of Fare")
plt.show()
```

Feature Encoding

Why Encode Categorical Data?

- Models work better with numbers than with text

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
# Label Encoding for 'sex' column
```

```
le = LabelEncoder()
```

```
df['sex_encoded'] = le.fit_transform(df['sex'])
```

```
# One-Hot Encoding for 'embark_town'
```

```
df = pd.get_dummies(df, columns=['embark_town'],
```

```
drop_first=True)
```

Feature Scaling

Why Scale Features?

Ensures all features have equal impact on the model

Standardization vs Normalization:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Standardization (Z-Score Normalization)
scaler = StandardScaler()
df[['age_scaled', 'fare_scaled']] =
scaler.fit_transform(df[['age', 'fare']])

# Min-Max Scaling
minmax_scaler = MinMaxScaler()
df[['age_minmax', 'fare_minmax']] =
minmax_scaler.fit_transform(df[['age', 'fare']])
```

Feature Engineering

Creating New Features

- **Family Size** = SibSp + Parch
- **Is Alone** = 1 if alone, else 0

Python Code for Feature Engineering:

```
# Creating new feature: Family Size  
df['family_size'] = df['sibsp'] + df['parch'] + 1  
  
# Creating new feature: Is Alone  
df['is_alone'] = (df['family_size'] == 1).astype(int)
```

Demo on EDA

Univariate Analysis

```
# Distribution of Age
plt.figure(figsize=(8,5))
sns.histplot(df['age'], kde=True, bins=30)
plt.title("Age Distribution")
plt.show()
```

```
# Countplot for categorical variables
plt.figure(figsize=(8,5))
sns.countplot(x=df['sex'])
plt.title("Gender Distribution")
plt.show()
```

Demo on EDA

Bivariate Analysis (Focusing on Survived Column)

```
# Survival rate by Gender
plt.figure(figsize=(8,5))
sns.barplot(x='sex', y='survived', data=df)
plt.title("Survival Rate by Gender")
plt.show()

# Survival rate by Passenger Class
plt.figure(figsize=(8,5))
sns.barplot(x='pclass', y='survived', data=df)
plt.title("Survival Rate by Passenger Class")
plt.show()

# Age distribution by survival
plt.figure(figsize=(8,5))
sns.histplot(df[df['survived'] == 1]['age'], kde=True, bins=30,
label='Survived', color='green', alpha=0.6)
sns.histplot(df[df['survived'] == 0]['age'], kde=True, bins=30, label='Not
Survived', color='red', alpha=0.6)
plt.legend()
plt.title("Age Distribution by Survival")
plt.show()
```

Conclusion

Key Steps in Data Preprocessing



1. Handle missing values to keep data complete
2. Remove outliers to avoid skewed results
3. Encode features to convert categories for modelling
4. Scale data for consistent numerical values
5. Engineer features to uncover useful insights
6. Use univariate and bivariate analysis to explore data and patterns

Q&A Session



Open floor for questions and clarifications.